

Trees & graphs

COMPLETE LEARNING GUIDE

What You'll Learn

Trees and graphs generalize the linear structures covered so far into branching and networked relationships, and traversing them correctly — knowing precisely when to reach for BFS versus DFS, and being able to implement both from memory — is one of the most frequently tested skills in placement interviews, since a huge share of "real" interview problems (file systems, social networks, dependency graphs, org charts) are trees or graphs in disguise. This guide covers binary tree structure and all four traversal orders, binary search tree (BST) properties and operations, a brief look at self-balancing trees, the two standard graph representations (adjacency list versus adjacency matrix) and when each is preferable, BFS and DFS on graphs with full Java implementations, and the classic applications — shortest path in an unweighted graph via BFS, and cycle detection — that come up constantly as standalone interview questions.

1. Binary Tree Structure and Traversal Orders

A binary tree node holds a value plus references to at most two children (left and right); traversal visits every node exactly once but in an order that depends on when a node is "visited" relative to its children. Inorder (left, node, right) visits a BST's nodes in sorted order — a uniquely important property. Preorder (node, left, right) visits the root first, useful for copying/serializing a tree's structure. Postorder (left, right, node) visits children before the parent, useful when children must be fully processed before the parent (like computing subtree sizes, or safely deleting a tree bottom-up). Level-order (breadth-first, level by level) uses a queue rather than the recursive call stack the other three implicitly rely on.

JAVA

```
class TreeNode {
    int val;
    TreeNode left, right;
    TreeNode(int val) { this.val = val; }
}

public static void inorder(TreeNode node, List<Integer> result) {
    if (node == null) return;          // base case
    inorder(node.left, result);
    result.add(node.val);               // visit AFTER left, BEFORE right
    inorder(node.right, result);
}

// For a valid BST, inorder traversal always produces sorted output -- O(n)
// time, O(h) space (h = height)
```

2. Binary Search Tree (BST) Properties and Operations

A BST maintains the invariant that every node's left subtree contains only smaller values and its right subtree contains only larger values, recursively, at every level — this property is what makes search, insertion, and deletion all achievable in $O(h)$ time, where h is the tree's height, by discarding half the remaining search space at each step (analogous to binary search on a sorted array). Critically, h equals $O(\log n)$ only for a balanced tree; a degenerate BST built by inserting already-sorted data collapses into a linked list, making $h = O(n)$ and every operation $O(n)$ — this degenerate case is exactly why self-balancing trees exist.

JAVA

```
// BST search -- O(h) time, O(1) space (iterative)
public static boolean search(TreeNode root, int target) {
    TreeNode current = root;
    while (current != null) {
        if (current.val == target) return true;
        current = target < current.val ? current.left : current.right;    //
    } // discard half the tree each step
    return false;
}
```

3. Self-Balancing Trees (Brief)

An AVL tree keeps every node's left and right subtree heights within 1 of each other, rebalancing via rotations after every insertion/deletion to guarantee $O(\log n)$ height at all times, making it slightly more rigidly balanced (and therefore faster for lookups) than a Red-Black tree, which allows more slack in exchange for cheaper rebalancing on writes. You are not expected to implement either from scratch in most interviews, but you should know they exist, why they exist (to prevent the degenerate $O(n)$ -height case), and that Java's `TreeMap`/`TreeSet` are themselves backed by a Red-Black tree internally — which is exactly why they guarantee $O(\log n)$ operations regardless of insertion order.

4. Graph Representations: Adjacency List vs. Adjacency Matrix

An adjacency list stores, for each vertex, a list of its directly connected neighbors — space-efficient at $O(V + E)$ for a graph with V vertices and E edges, and efficient for iterating a vertex's neighbors (exactly its degree), making it the default choice for most graph problems, especially sparse graphs. An adjacency matrix stores a $V \times V$ grid where `matrix[i][j]` indicates whether an edge exists between vertex i and j — $O(V^2)$ space regardless of how many edges actually exist, but $O(1)$ to check whether a specific edge exists, which an adjacency list can only answer in $O(\text{degree})$ time. The practical rule: use adjacency list for sparse graphs and most traversal-based problems; use adjacency matrix when frequent $O(1)$ edge-existence checks matter more than memory, or the graph is dense (E close to V^2).

JAVA

```
// Adjacency list -- the default choice for most problems
Map<Integer, List<Integer>> graph = new HashMap<>();
graph.computeIfAbsent(0, k -> new ArrayList<>()).add(1);    // edge 0 -> 1
graph.computeIfAbsent(0, k -> new ArrayList<>()).add(2);    // edge 0 -> 2
```

5. BFS: Level-by-Level Traversal

Breadth-first search explores a graph level by level, visiting all neighbors of the current node before moving to any of their neighbors, implemented with a queue and a visited set to avoid revisiting nodes (essential in graphs, which — unlike trees — can contain cycles). BFS is the standard algorithm for finding the shortest path in an unweighted graph, because the first time BFS reaches a given node is guaranteed to be via a shortest possible path from the source, a guarantee DFS does not provide.

JAVA

```
// BFS -- O(V + E) time, O(V) space
public static List<Integer> bfs(Map<Integer, List<Integer>> graph, int start) {
    List<Integer> order = new ArrayList<>();
    Set<Integer> visited = new HashSet<>();
    Queue<Integer> queue = new ArrayDeque<>();

    queue.offer(start);
    visited.add(start);

    while (!queue.isEmpty()) {
        int node = queue.poll();
        order.add(node);
        for (int neighbor : graph.getOrDefault(node, List.of())) {
            if (!visited.contains(neighbor)) {
                visited.add(neighbor);    // mark visited AT ENQUEUE time, not at
dequeued time        queue.offer(neighbor);
            }
        }
    }
    return order;
}
```

6. DFS: Depth-First Exploration

Depth-first search explores as far as possible down one path before backtracking, implemented either recursively (using the call stack implicitly) or iteratively with an explicit stack — both visit the same set of nodes overall but in a different order than BFS. DFS is preferred for problems about a path's existence or structure (does a path exist at all, topological sorting, detecting a cycle, exploring all of a connected component) rather than shortest-path problems, where BFS's level-by-level guarantee is what's actually needed.

JAVA

```
// DFS (recursive) -- O(V + E) time, O(V) space (including call stack)
public static void dfs(Map<Integer, List<Integer>> graph, int node, Set<Integer>
visited, List<Integer> order) { return;
    if (visited.contains(node)) { return;
        visited.add(node);
        order.add(node);
        for (int neighbor : graph.getOrDefault(node, List.of())) {
            dfs(graph, neighbor, visited, order);
        }
    }
}
```

7. Cycle Detection in a Graph

Detecting a cycle in an undirected graph via DFS checks, for each visited neighbor, whether it's the node's immediate parent (which is expected and not a cycle) or some other already-visited node (which IS a cycle, since reaching an already-visited non-parent node means there are two distinct paths to it). Detecting a cycle in a directed graph is different and requires tracking nodes currently "in progress" on the current DFS path (not just visited overall) — reaching a node that's in-progress on the current path (not merely visited previously via a different path) is what actually proves a directed cycle, since visiting an already-fully-processed node elsewhere in the graph is completely normal in a DAG.

Common Pitfalls

- Using DFS to find a shortest path in an unweighted graph — only BFS guarantees the first-reached path is shortest; DFS gives no such guarantee.
- Forgetting the visited set in a graph traversal (fine to omit in a tree, which has no cycles), causing infinite loops on a graph that contains a cycle.
- Marking a node visited at dequeue time instead of enqueue time in BFS, which can enqueue the same node multiple times before it's first processed.
- Assuming BST operations are $O(\log n)$ unconditionally, forgetting a degenerate (unbalanced) BST built from sorted input collapses to $O(n)$ height.
- Confusing undirected-graph cycle detection (check for a non-parent visited neighbor) with directed-graph cycle detection (check for a node currently in-progress on the DFS path).
- Choosing an adjacency matrix for a large, sparse graph, wasting $O(V^2)$ space when an adjacency list would use only $O(V + E)$.
- Forgetting that a recursive DFS's space complexity includes call-stack depth (up to $O(V)$ in the worst case, e.g. a long path), not just the explicit visited set.

Key Takeaways

- Inorder traversal of a BST produces sorted output — a uniquely useful property among the four traversal orders.

- BST search/insert/delete are $O(h)$; h is $O(\log n)$ only when the tree is balanced — a degenerate BST from sorted input is $O(n)$ height.
- Adjacency list ($O(V+E)$ space) suits sparse graphs and most traversal problems; adjacency matrix ($O(V^2)$ space, $O(1)$ edge lookup) suits dense graphs or frequent edge-existence checks.
- BFS guarantees the shortest path in an unweighted graph; DFS does not — pick the traversal based on what the problem actually needs.
- Both BFS and DFS run in $O(V + E)$ time; always track visited nodes explicitly in a graph to avoid infinite loops from cycles.
- Directed-graph cycle detection needs an "in current path" marker distinct from a general "visited" marker; undirected-graph cycle detection just needs to exclude the immediate parent.

Learn More

GeeksforGeeks — Tree Traversals (Inorder, Preorder, Postorder)

<https://www.geeksforgeeks.org/dsa/tree-traversals-inorder-preorder-and-postorder/>

GeeksforGeeks — BFS vs DFS

<https://www.geeksforgeeks.org/dsa/bfs-vs-dfs-for-binary-tree/>

GeeksforGeeks — Detect Cycle in a Graph

<https://www.geeksforgeeks.org/dsa/detect-cycle-in-a-graph/>